

JURNAL MODUL 7

PRAKTIKUM SISTEM OPERASI



**Universitas
Telkom**

Disusun Oleh :
Nama : Ganes Gemi Putra
Kelas : SE-07-02
NIM : 2311104075

Dosen Pengampu : Yudha Islami Sulistya, S.Kom., M.Cs.

**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY
PURWOKERTO
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sistem operasi merupakan perangkat lunak yang bertugas mengelola sumber daya komputer serta menyediakan layanan bagi program lain yang berjalan di atasnya. Pada sistem yang mendukung multitasking, beberapa proses dapat berjalan secara bersamaan sehingga dibutuhkan mekanisme sinkronisasi agar proses-proses tersebut tidak saling mengganggu ketika mengakses data atau sumber daya yang sama.

Salah satu mekanisme sinkronisasi yang umum digunakan adalah semaphore. Semaphore berfungsi sebagai penanda dan pengatur giliran eksekusi proses, sehingga urutan kerja antarproses dapat dikendalikan secara aman. Pada praktikum Modul 7 ini, konsep semaphore diterapkan di sistem operasi Xinu melalui dua bentuk permasalahan, yaitu pengaturan urutan tiga proses dan sinkronisasi pada kasus producer-consumer.

Melalui praktikum ini, mahasiswa tidak hanya memahami konsep semaphore secara teori, tetapi juga dapat mengimplementasikannya secara langsung ke dalam source code, melakukan kompilasi, menjalankan program, serta menganalisis hasil keluaran yang dihasilkan oleh sistem.

1.2 Rumusan Masalah

Rumusan masalah pada praktikum ini adalah bagaimana cara menerapkan semaphore untuk mengatur sinkronisasi beberapa proses pada sistem operasi Xinu serta bagaimana memastikan setiap proses berjalan sesuai urutan yang diinginkan tanpa menimbulkan konflik akses data.

1.3 Tujuan Praktikum

Tujuan dari praktikum Modul 7 adalah untuk memahami konsep semaphore, mengimplementasikan semaphore pada program berbasis Xinu, mengatur sinkronisasi tiga proses pada soal pertama, serta menerapkan mekanisme sinkronisasi produser dan konsumen pada soal kedua.

1.4 Manfaat Praktikum

Manfaat dari praktikum ini adalah mahasiswa memperoleh pengalaman langsung dalam menulis program konkuren, memahami pentingnya sinkronisasi, mengetahui peran mutex dan semaphore dalam pengendalian proses, serta mampu menganalisis hasil eksekusi program secara sistematis dan profesional.

BAB II

DASAR TEORI

2.1 Proses dan Konkruensi

Proses adalah program yang sedang dieksekusi. Dalam sistem operasi, beberapa proses dapat berjalan secara konkuren, yaitu seolah-olah berlangsung bersamaan. Keadaan ini menuntut adanya pengaturan urutan kerja agar hasil eksekusi tetap benar dan tidak saling bertabrakan.

2.2 Semaphore

Semaphore adalah variabel sinkronisasi yang digunakan untuk mengatur akses proses terhadap sumber daya tertentu atau untuk memberikan sinyal bahwa suatu proses telah menyelesaikan pekerjaannya. Pada Xinu, operasi utama semaphore adalah `wait()` untuk menunggu giliran dan `signal()` untuk membangunkan atau memberi izin kepada proses lain.

2.3 Mutex

Mutex adalah mekanisme penguncian yang digunakan untuk menjamin bahwa hanya satu proses saja yang dapat memasuki critical section pada satu waktu. Dalam kasus producer-consumer, mutex digunakan agar data bersama tidak diakses secara bersamaan oleh produser dan konsumen.

2.4 Producer-Consumer

Producer-consumer merupakan salah satu permasalahan sinkronisasi klasik. Produser bertugas menghasilkan data, sedangkan konsumen mengambil dan menggunakan data tersebut. Agar data yang dibaca konsumen selalu benar, diperlukan semaphore untuk menandai kondisi data kosong atau penuh serta mutex untuk menjaga critical section.

2.5 Semaphore pada Xinu

Pada sistem operasi Xinu, semaphore diinisialisasi dengan fungsi `semcreate()`. Setelah itu, setiap proses dapat disinkronkan dengan `wait()` dan `signal()`. Pada praktikum ini digunakan tiga semaphor untuk mengatur tiga proses pada soal pertama, sedangkan pada soal kedua digunakan satu mutex dan dua semaphore, yaitu `empty` dan `full`.

BAB III

METODOLOGI DAN LANGKAH KERJA

3.1 Perangkat dan Lingkungan Kerja

Praktikum dikerjakan menggunakan lingkungan virtualisasi Oracle VM VirtualBox, sistem operasi Debian pada development system, editor gedit untuk menulis source code, compiler bawaan Xinu pada direktori compile, serta minicom untuk menampilkan output program yang dijalankan pada backend.

Komponen	Keterangan
Virtualisasi	Oracle VM VirtualBox
Development VM	Debian 64-bit
Editor	gedit
Bahasa Pemrograman	C pada Xinu
Direktori source code	~/xinu/system/main.c
Direktori kompilasi	~/xinu/compile
Media output	minicom dan backend Xinu

3.2 Langkah Kerja Umum

Secara umum, langkah kerja praktikum dilakukan mulai dari membuka file main.c pada direktori system, menuliskan source code sesuai kebutuhan soal, menyimpan perubahan, melakukan kompilasi pada direktori compile, kemudian menjalankan program menggunakan minicom dan backend untuk melihat hasil output.

```
cd ~/xinu/system
gedit main.c
cd ~/xinu/compile
make clean
make
sudo minicom --color=on
```

3.3 Langkah Penyelesaian Soal Nomor 1

Pada soal nomor 1 dibuat tiga proses, yaitu proses A, proses B, dan proses C. Masing-masing proses bertugas menampilkan teks “kwak”, “kwik”, dan “kwek”. Tiga semaphore digunakan untuk mengatur giliran proses agar urutan output selalu berulang secara tetap.

Strategi yang digunakan adalah menjadikan semaphore pertama bernilai awal 1 sebagai proses pembuka, sedangkan dua semaphore lainnya bernilai 0 agar proses lain menunggu. Setelah suatu proses selesai mencetak teks, proses tersebut akan memanggil signal() untuk membangunkan proses berikutnya.

3.4 Langkah Penyelesaian Soal Nomor 2

Pada soal nomor 2 dibuat dua proses, yaitu produser dan konsumen. Produser menghasilkan nilai bilangan dari 1 sampai 1000, sedangkan konsumen menampilkan nilai yang telah dihasilkan. Satu mutex digunakan untuk mengunci akses ke data bersama, sementara semaphore empty dan full digunakan untuk mengatur kondisi data kosong dan penuh.

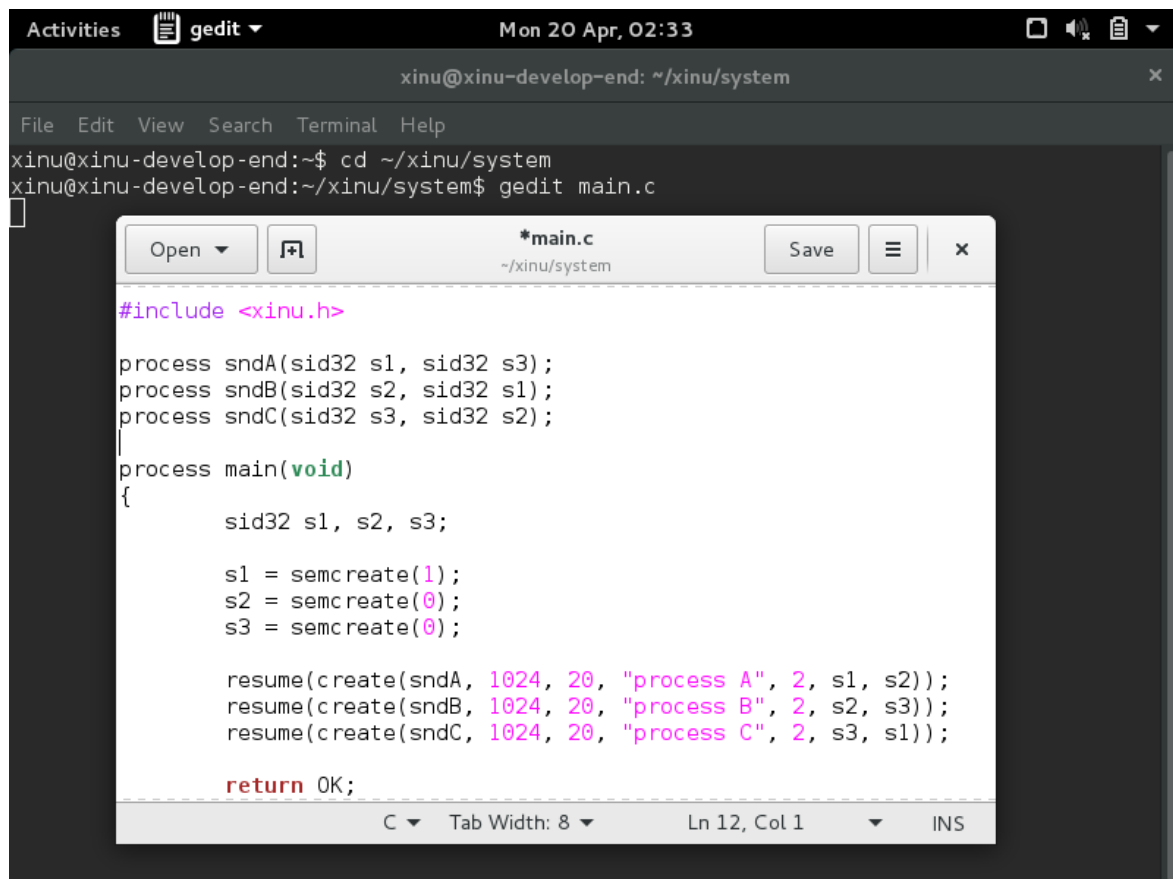
Dengan rancangan ini, produser hanya dapat menulis ketika buffer kosong, dan konsumen hanya dapat membaca ketika data telah tersedia. Mekanisme tersebut memastikan urutan data tetap benar dan tidak terjadi bentrok akses.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi dan Hasil Soal Nomor 1

Implementasi nomor 1 menunjukkan penggunaan tiga proses dan tiga semaphore. Pada bagian fungsi main, masing-masing semaphore diinisialisasi dengan nilai awal tertentu, lalu ketiga proses dijalankan menggunakan `create()` dan `resume()`.



```
Activities gedit Mon 20 Apr, 02:33
xinu@xinu-develop-end: ~/xinu/system
File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ cd ~/xinu/system
xinu@xinu-develop-end:~/xinu/system$ gedit main.c

*main.c
~/xinu/system

#include <xinu.h>

process sndA(sid32 s1, sid32 s3);
process sndB(sid32 s2, sid32 s1);
process sndC(sid32 s3, sid32 s2);
|
process main(void)
{
    sid32 s1, s2, s3;

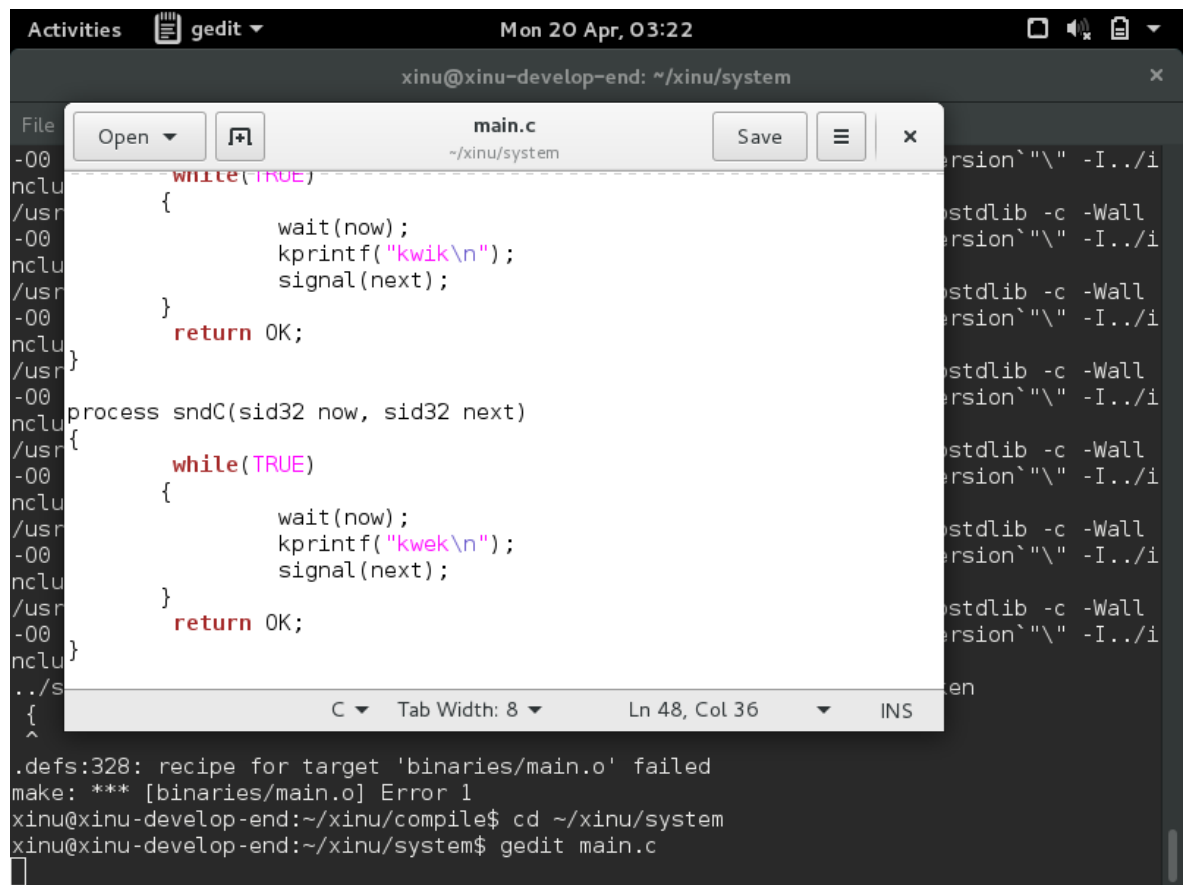
    s1 = semcreate(1);
    s2 = semcreate(0);
    s3 = semcreate(0);

    resume(create(sndA, 1024, 20, "process A", 2, s1, s2));
    resume(create(sndB, 1024, 20, "process B", 2, s2, s3));
    resume(create(sndC, 1024, 20, "process C", 2, s3, s1));

    return OK;
}
```

Gambar 1. Source code nomor 1 pada bagian deklarasi proses dan fungsi main

Pada bagian fungsi proses, setiap proses melakukan `wait()` pada semaphore miliknya, menampilkan teks sesuai tugasnya, kemudian memanggil `signal()` untuk mengaktifkan proses berikutnya. Pola ini membentuk siklus A ke B ke C secara berulang.



```

Activities gedit Mon 20 Apr, 03:22
xinu@xinu-develop-end: ~/xinu/system

File Edit View Window Help
main.c ~/xinu/system Save
while(TRUE)
{
    wait(now);
    kprintf("kwik\n");
    signal(next);
}
return OK;

process sndC(sid32 now, sid32 next)
{
    while(TRUE)
    {
        wait(now);
        kprintf("kwek\n");
        signal(next);
    }
    return OK;
}

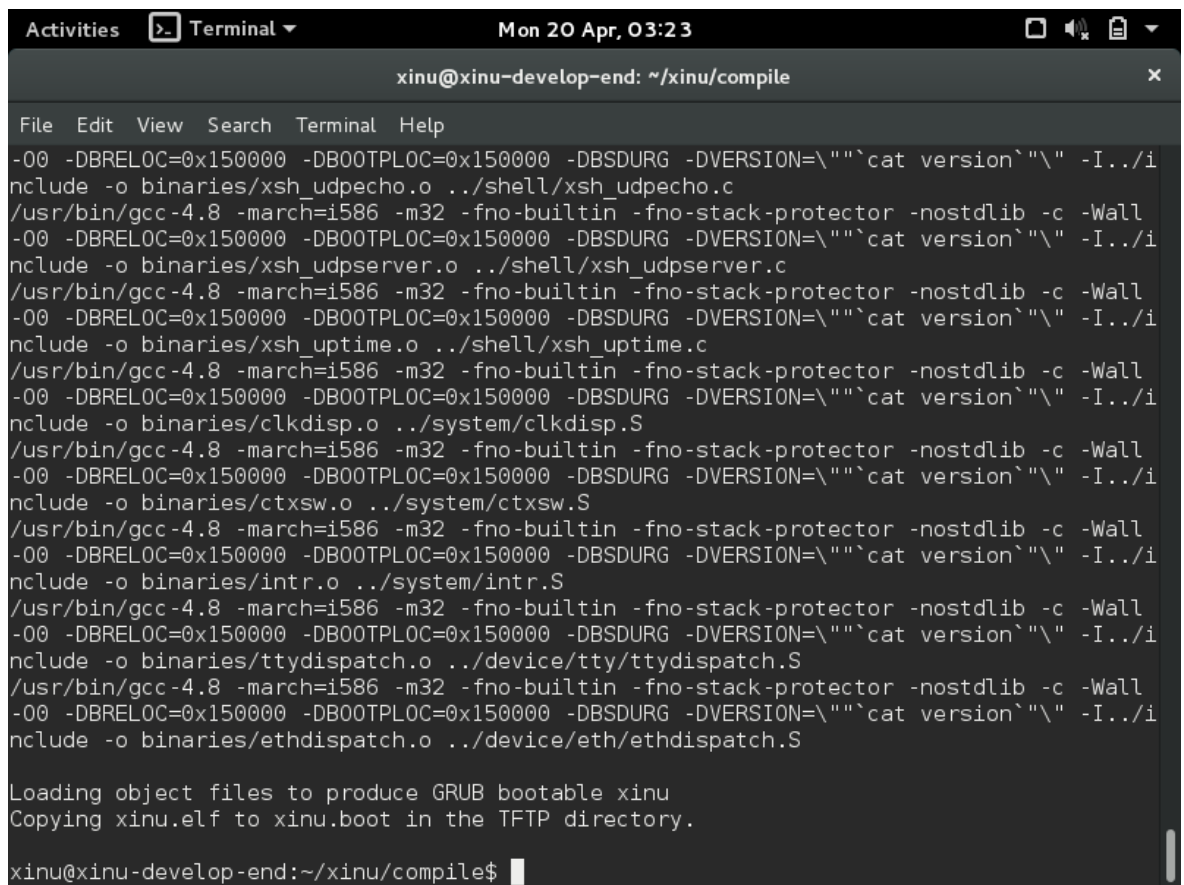
..s
{
    C Tab Width: 8 Ln 48, Col 36 INS

.defs:328: recipe for target 'binaries/main.o' failed
make: *** [binaries/main.o] Error 1
xinu@xinu-develop-end:~/xinu/compile$ cd ~/xinu/system
xinu@xinu-develop-end:~/xinu/system$ gedit main.c

```

Gambar 2. Source code nomor 1 pada bagian fungsi proses

Setelah source code selesai ditulis, program dikompilasi pada direktori compile. Hasil kompilasi menunjukkan bahwa objek berhasil disusun dan file xinu.boot berhasil dibuat tanpa error.



```

Activities Terminal Mon 20 Apr, 03:23
xinu@xinu-develop-end: ~/xinu/compile

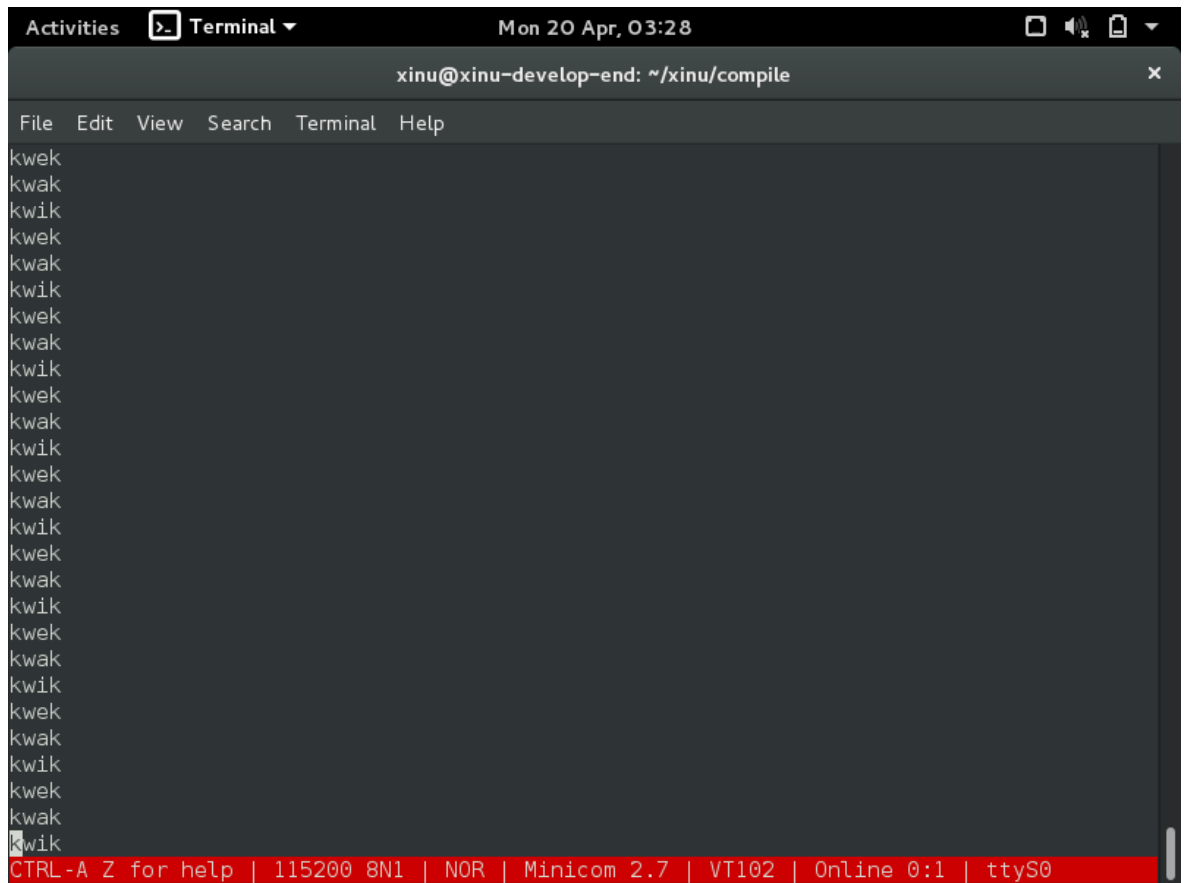
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"cat version\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
  
```

Gambar 3. Hasil kompilasi program nomor 1

Program kemudian dijalankan melalui minicom. Output yang muncul memperlihatkan urutan “kwak”, “kwik”, dan “kwek” secara berulang. Hal ini menunjukkan semaphore bekerja dengan baik untuk mengatur urutan tiga proses.



A terminal window titled 'Terminal' with a dropdown arrow. The window shows the prompt 'xinu@xinu-develop-end: ~/xinu/compile'. The output consists of a sequence of 'kwek' and 'kwik' messages. The terminal status bar at the bottom is red and contains the text: 'CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:1 | ttyS0'.

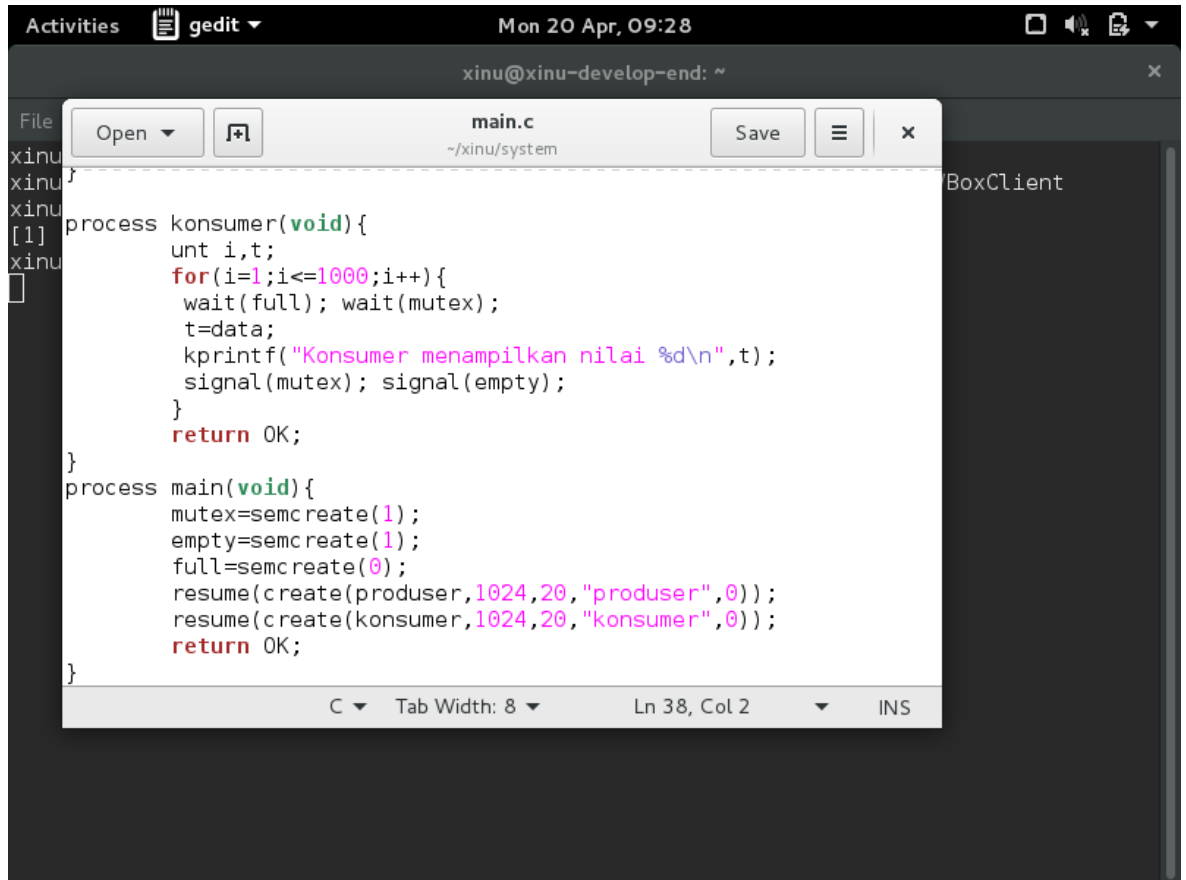
```
Activities Terminal Mon 20 Apr, 03:28
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
kwek
kwak
kwik
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:1 | ttyS0
```

Gambar 4. Output program nomor 1

Berdasarkan hasil tersebut, dapat disimpulkan bahwa tujuan soal nomor 1 telah tercapai. Tiga proses berhasil berjalan secara konkuren namun tetap terkendali urutannya karena masing-masing proses hanya aktif ketika memperoleh sinyal dari semaphore yang sesuai.

4.2 Implementasi dan Hasil Soal Nomor 2

Implementasi nomor 2 menerapkan pola producer-consumer. Pada source code terlihat penggunaan proses produser dan konsumen, variabel data bersama, satu mutex, serta dua semaphore bernama empty dan full. Produser bertugas mengisi data, sedangkan konsumen membacanya secara bergantian.



```

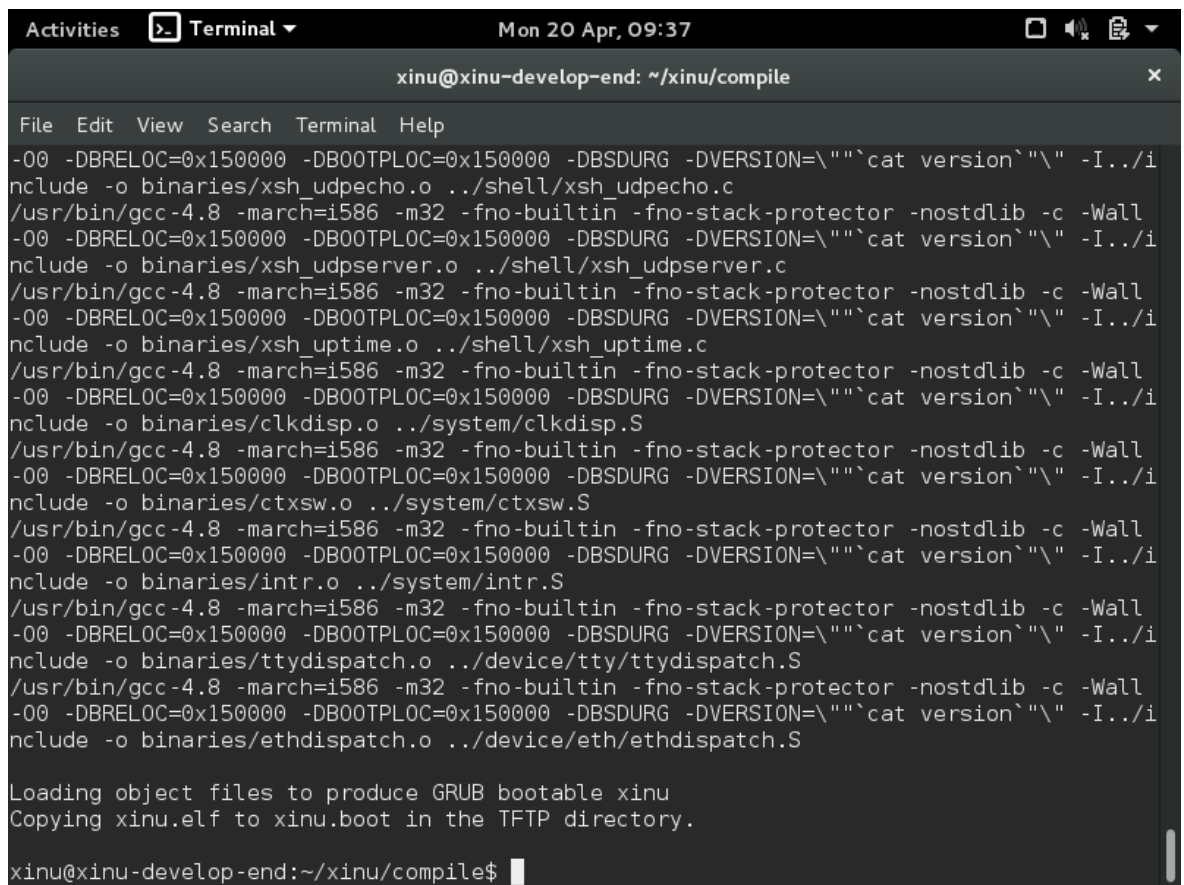
process konsumer(void){
    unt i,t;
    for(i=1;i<=1000;i++){
        wait(full); wait(mutex);
        t=data;
        kprintf("Konsumer menampilkan nilai %d\n",t);
        signal(mutex); signal(empty);
    }
    return OK;
}

process main(void){
    mutex=semcreate(1);
    empty=semcreate(1);
    full=semcreate(0);
    resume(create(produser,1024,20,"produser",0));
    resume(create(konsumer,1024,20,"konsumer",0));
    return OK;
}

```

Gambar 5. Source code nomor 2

Pada tahap kompilasi, program berhasil dibangun tanpa error sehingga dapat dijalankan pada backend Xinu. Hal ini menunjukkan bahwa struktur program, deklarasi proses, serta penggunaan semaphore sudah sesuai dengan yang dibutuhkan oleh sistem.



```

Activities  Terminal  Mon 20 Apr, 09:37
xinu@xinu-develop-end: ~/xinu/compile

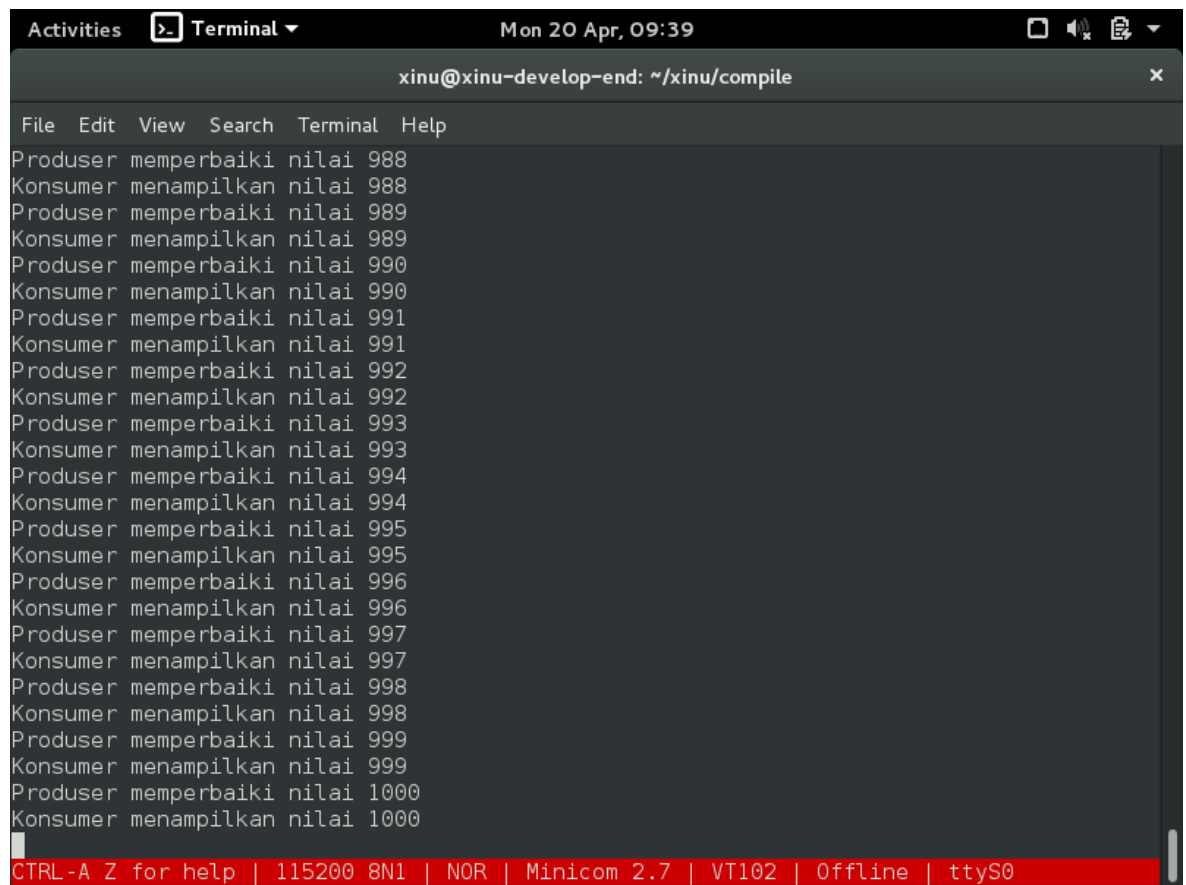
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\"\"cat version\"\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$
  
```

Gambar 6. Hasil kompilasi program nomor 2

Hasil output program memperlihatkan bahwa nilai diproses secara berurutan. Produser menghasilkan nilai, kemudian konsumen menampilkan nilai yang sama. Pada dokumentasi yang dilampirkan, urutan nilai berjalan konsisten hingga mencapai 1000, sehingga logika sinkronisasi berhasil diterapkan.



```
Activities Terminal Mon 20 Apr, 09:39
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
Produser memperbaiki nilai 988
Konsumer menampilkan nilai 988
Produser memperbaiki nilai 989
Konsumer menampilkan nilai 989
Produser memperbaiki nilai 990
Konsumer menampilkan nilai 990
Produser memperbaiki nilai 991
Konsumer menampilkan nilai 991
Produser memperbaiki nilai 992
Konsumer menampilkan nilai 992
Produser memperbaiki nilai 993
Konsumer menampilkan nilai 993
Produser memperbaiki nilai 994
Konsumer menampilkan nilai 994
Produser memperbaiki nilai 995
Konsumer menampilkan nilai 995
Produser memperbaiki nilai 996
Konsumer menampilkan nilai 996
Produser memperbaiki nilai 997
Konsumer menampilkan nilai 997
Produser memperbaiki nilai 998
Konsumer menampilkan nilai 998
Produser memperbaiki nilai 999
Konsumer menampilkan nilai 999
Produser memperbaiki nilai 1000
Konsumer menampilkan nilai 1000

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

Gambar 7. Output akhir program nomor 2

Dari hasil tersebut dapat dilihat bahwa semaphore empty dan full berhasil mengatur kapan produser boleh menulis dan kapan konsumen boleh membaca. Sementara itu, mutex berperan menjaga critical section agar data tidak diakses secara bersamaan. Dengan demikian, program telah memenuhi konsep sinkronisasi producer-consumer menggunakan semaphore pada Xinu.

4.3 Analisis Hasil Praktikum

Secara keseluruhan, kedua soal pada Modul 7 menunjukkan bahwa semaphore sangat efektif digunakan untuk sinkronisasi proses. Pada soal pertama, semaphore mengatur urutan proses yang sifatnya bergilir. Pada soal kedua, semaphore dan mutex dipadukan untuk menangani data bersama secara aman. Hasil praktikum juga menunjukkan bahwa rancangan konkruensi harus selalu mempertimbangkan urutan eksekusi, critical section, dan komunikasi antarpromses.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa semaphore pada Xinu berhasil diimplementasikan untuk menyelesaikan dua jenis permasalahan sinkronisasi. Soal nomor 1 berhasil menampilkan urutan tiga proses secara berulang dan teratur, sedangkan soal nomor 2 berhasil mengimplementasikan mekanisme producer-consumer dengan hasil keluaran yang konsisten sampai bilangan 1000.

Praktikum ini memberikan pemahaman bahwa sinkronisasi merupakan komponen penting dalam sistem operasi, khususnya ketika beberapa proses berjalan secara bersamaan. Tanpa adanya semaphore dan mutex, urutan proses dapat menjadi tidak terkontrol dan data bersama berpotensi mengalami konflik akses.

5.2 Saran

Untuk praktikum selanjutnya, mahasiswa disarankan lebih memahami konsep sinkronisasi sebelum mulai menulis source code, melakukan dokumentasi setiap tahap pengerjaan secara rapi, serta mengecek kembali hasil output agar sesuai dengan format yang diminta pada modul. Selain itu, pemahaman tentang hubungan antara proses, semaphore, dan critical section perlu terus dilatih agar implementasi program menjadi lebih baik.

Daftar Pustaka

Modul 7 Praktikum Sistem Operasi: Semaphore.

Petunjuk dan hint praktikum mengenai inisiasi semaphore serta penggunaan 3 proses, 3 semaphore, 1 mutex, dan 2 semaphore pada kasus producer-consumer.